

TEORÍA DE ALGORITMOS
CURSO ECHEVARRÍA

Trabajo Práctico 1

2° Cuatrimestre 2025

Grupo 08

Damaris Juares
108566

Denise Dall Acqua
108645

Kevin Carbajal
108304

Rodriguez Nazareno Jose Luis
101891

Facundo Monpelat
92716

Índice

1. Problema 1	4
1.1. Análisis	4
1.1.1. Preamálisis	4
1.1.2. Análisis Recursivo	4
1.2. Diseño	6
1.2.1. Pseudocódigo	6
1.2.2. Estructuras	6
1.3. Seguimiento	7
1.4. Complejidad	8
1.5. Set de Datos	8
1.6. Tiempos de ejecución	8
1.7. Informe de Resultados	9
2. Problema 2	10
2.1. Analisis	10
2.2. Diseño	11
2.3. Seguimiento del Algoritmo	12
2.4. Análisis de Complejidad Temporal	13
2.5. Set de datos	13
2.6. Tiempos de ejecucion	14
2.7. Ajuste con función cuadrática	15
2.8. Error cuadrático del ajuste	15
3. Problema 3	17
3.1. Análisis	17
3.1.1. Supuestos, condiciones y limitaciones	17
3.1.2. Descripción de las podas que realiza el algoritmo	17
3.1.3. Análisis del tamaño del espacio de soluciones factibles	17
3.2. Diseño	18
3.2.1. Pseudocódigo	18
3.2.2. Estructuras de datos utilizadas	18
3.3. Seguimiento	19
3.4. Complejidad	19
3.5. Sets de Datos	19
3.6. Tiempos de ejecución	20
4. Problema 4	22
4.1. Análisis	22
4.1.1. Supuestos, condiciones y limitaciones	22
4.1.2. Ecuación de Recurrencia	22

4.1.3.	Justificación del cumplimiento de los requisitos de Subestructura Óptima y Subproblemas Superpuestos	22
4.1.4.	Indicación de cómo usa Memoization	22
4.2.	Diseño	23
4.2.1.	Pseudocódigo	23
4.2.2.	Estructuras de datos utilizadas	24
4.3.	Seguimiento	24
4.4.	Complejidad	25
4.5.	Sets de Datos	25
4.6.	Tiempos de ejecución	26
4.7.	Informe de Resultados	27
4.7.1.	Comparación de tiempos con la complejidad temporal teorica	27
4.7.2.	Comparación con el Problema 3	27

1. Problema 1

1.1. Análisis

1.1.1. Preamálisis

- Supuestos

Se considera posible también índices negativos ya que puede darse el caso de que $A[-i] = -i$ siendo A un array de valores enteros que incluye este tipo de valores (negativos).
Ejemplo: $A = [-3, -2, -1]$ $A[-1] = -1$.

- Condiciones

Al ser un array de enteros diferentes no se puede dar el caso en que $A[i] = j$ y $A[j] = i$.
Ejemplo: $A[4] = 9$ y $A[9] = 9$ no es posible.

Al ser un array ordenado, para cada índice i , dado un índice i si $A[i] > i$ los índices desde i hasta $A[i]$ no cumplen lo pedido. Ejemplo: $A[4] = 9$ No puede darse $A[5] = 5$ porque los valores están ordenados y el elemento anterior tenía un valor de 9. Tampoco puede darse $A[A[i]] = j$, ej $A[A[4]] = 9$ con $A[4] = 9$ porque no hay valores repetidos.

Esto es, el intervalo desde $[i, A[i]] = [4, 9]$ no cumplen lo pedido.

Pero puede darse $A[3] = 3$ El subintervalo posible es $[inicio, i-1]$ y $[A[i]+1, final]$.

- Primicias

Al ser un arreglo ordenado el problema se asemeja a un problema en el que es posible buscar por partes subdividiendo en cada iteración el array en subintervalos posibles.

1.1.2. Análisis Recursivo

La estrategia es simple: verificar si el elemento a la mitad del conjunto cumple lo pedido, o sino seguir buscando en subintervalos posibles. En el primer caso terminamos el proceso. El proceso continúa hasta encontrar un elemento que cumpla lo pedido o - en el peor caso - hasta agotar todos las subdivisiones de intervalos posibles.

- Caso Base

Llegaremos al caso base cuando ya no quedan elementos. Es decir

$$inicio > final$$

Siendo inicio y final los límites del intervalo que se está analizando.

- Casos Recursivos

Cada nuevo subproblema contendrá como mucho la mitad de los elementos (contemplando solo los subintervalos posibles que cumplan lo pedido).

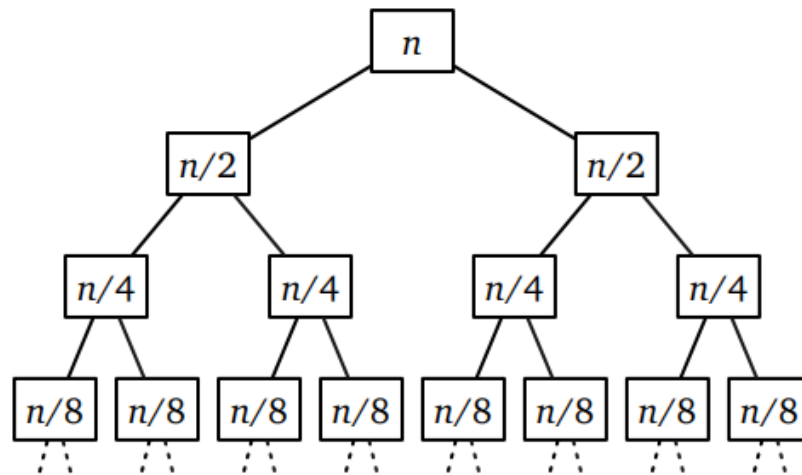


Figura 1: Árbol de Recursión

En el análisis vemos que tendremos “k” subproblemas por las llamadas recursivas. Llegaremos al caso base donde ya no quedan elementos.

Los resultados parciales se combinan de manera que cada llamada recursiva valida si el subproblema resuelto devolvió un resultado válido (es decir, distinto a “no existe i”). Es decir:

Primero se hace una llamada recursiva para un subintervalo. Se guarda el resultado en una variable (por ejemplo, ‘result’).

Si ‘result’ es válido, se retorna ese resultado inmediatamente. Si ‘result’ indica que no se encontró la solución, se procede a evaluar el otro subintervalo y se retorna ese resultado.

Esta estrategia permite que cada llamada recursiva combine los resultados parciales de sus subproblemas, propagando la solución hacia arriba solo si se ha encontrado un resultado válido

1.2. Diseño

1.2.1. Pseudocódigo

```
1 Sea L una lista de n elementos ordenados de menor a mayor
2
3 problema1(L, inicio, final):
4     Si inicio > final:
5         Retornar No existe el elemento
6
7     Sea mitad el valor intermedio entre inicio y fin
8
9     Si lista[mitad] == mitad:
10        return mitad
11
12    Si lista[mitad] > mitad:
13        result = problema1(lista, inicio, mitad - 1)
14        if result != "no existe i":
15            return result
16        return problema1(lista, lista[mitad] + 1, final)
17
18    Osino:
19        result = problema1(lista, inicio, mitad - 1)
20        Si result != "no existe i":
21            return result
22        return problema1(lista, mitad + 1, final)
```

Listing 1: Pseudocódigo de la solución propuesta al problema 1

1.2.2. Estructuras

Solo manipula la lista original mediante índices y variables locales. No crea ni utiliza listas, diccionarios u otros contenedores para almacenar información intermedia. Opera directamente sobre el arreglo dado, lo que implica que la memoria de trabajo se limita a variables simples (como 'inicio', 'final' y 'mitad') y no se generan copias o estructuras auxiliares.

1.3. Seguimiento

El comportamiento de la solución ante un array A de ejemplo, con $A = [0, 3, 4, 5, 6, 9]$, es el siguiente:

A continuación se presenta el seguimiento detallado de la ejecución de la función `problema1` para la llamada:

`problema1([0, 3, 4, 5, 6, 9], 0, 5)`

1. Primera llamada:

- Parámetros: inicio = 0, final = 5.
- Se calcula la posición de la mitad:

$$\text{mitad} = 0 + \frac{5 - 0}{2} = 2.$$

- Se evalúa: `lista[2] = 4`; como $4 \neq 2$, falla la condición de igualdad.
- Se evalúa la condición:

`if lista[2] > 2` (se cumple, ya que $4 > 2$).

- Se realiza la llamada recursiva:

`result = problema1(lista, 0, 1).`

2. Segunda llamada (recursiva):

- Parámetros: inicio = 0, final = 1.
- Se calcula la posición de la mitad:

$$\text{mitad} = 0 + \frac{1 - 0}{2} = 0.$$

- Se evalúa: `lista[0] = 0`; se cumple la condición:

`if lista[mitad] == mitad,`

pues $0 = 0$.

- Se retorna el valor 0.

3. Retorno a la primera llamada:

- La variable `result` recibe el valor 0.
- Se verifica:

`if result != 'no existe i'.`

- Dado que $0 \neq \text{"no existe i"}$, se retorna 0.

4. Salida final:

- La función retorna 0, que es lo que se imprime en la llamada principal.

1.4. Complejidad

Utilizamos una relación de recurrencia $T(n)$ cuyo parámetro de entrada es el tamaño del conjunto. El problema se divide en dos subproblemas de la mitad del tamaño actual. La complejidad de cada uno es $\mathcal{O}(\log n)$. Luego se realizan un número constante de operaciones de comparación. Estas operaciones no dependen del tamaño del conjunto y por lo tanto son $\mathcal{O}(1)$ en su conjunto. Por lo tanto podemos escribir la recurrencia como

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(1) \quad (1)$$

con el caso base de

$$T(1) = \mathcal{O}(1) \quad (2)$$

En conclusión, la complejidad temporal del algoritmo planteado es

$$\mathcal{O}(\log n)$$

1.5. Set de Datos

Se utiliza un archivo .txt pre-generado para ejecutar las pruebas correspondientes de la solución propuesta, teniendo en cuenta diferentes tamaños de array y diferentes posiciones del elemento buscado (si lo hay), para poder verificar gráficamente la respuesta temporal del algoritmo antes tamaños variables de n .

1.6. Tiempos de ejecución

A continuación se presenta la medición y gráfica de los tiempos de ejecución para los tamaños de arrays: 40, 100, 200, 500 y 1000.

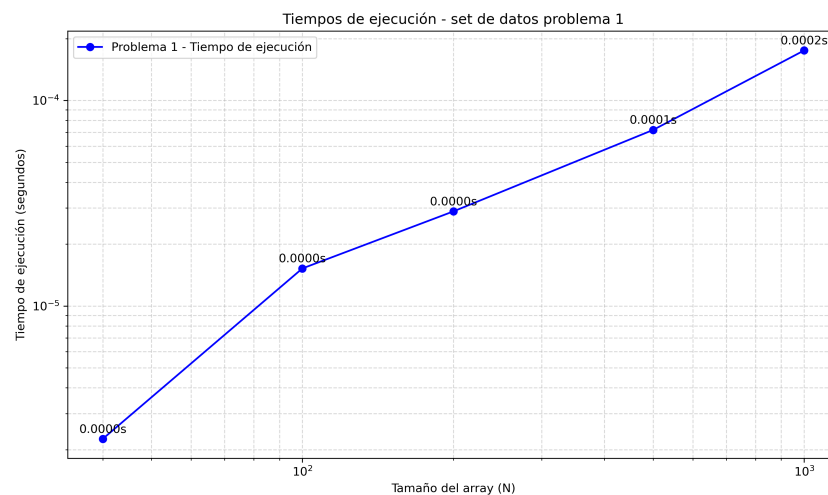


Figura 2: Mediciones temporales

1.7. Informe de Resultados

Se había estimado que el algoritmo tiene una *complejidad temporal* de $O(\log n)$, lo cual es consistente con el comportamiento observado en el gráfico:

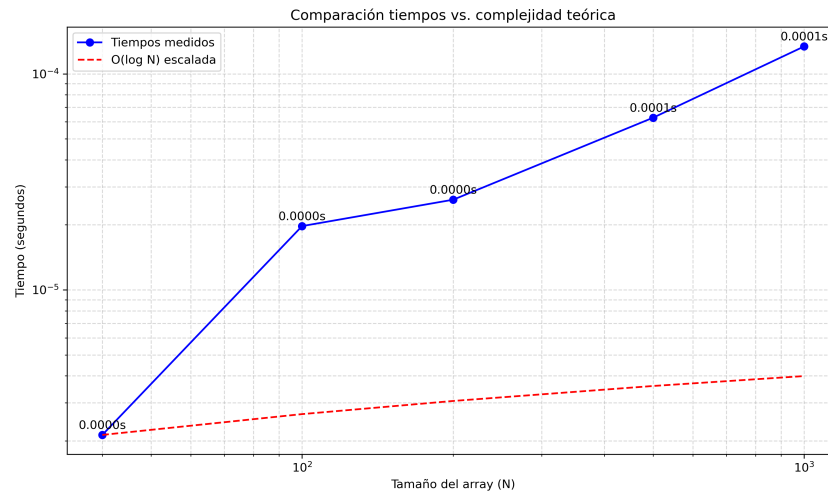


Figura 3: Mediciones temporales vs Mediciones Complejidad

- **Comparación con la complejidad estimada** La forma casi lineal en el gráfico (cuando el eje Y es logarítmico) sugiere que el crecimiento no es cuadrático ($O(n^2)$) ni lineal puro ($O(n)$), sino sublineal respecto a un cuadrado, como $O(\log n)$.
- **Conclusiones** El algoritmo optimizado muestra una escalabilidad eficiente, manteniendo tiempos bajos incluso para arrays de tamaños grandes.

2. Problema 2

Se tiene un arreglo A de tamaño n , conformado por enteros diferentes (positivos, negativos o cero), y que no está ordenado.

Se define un **intervalo positivo** como un subarreglo $A[i..j]$ con $i \neq j$ tal que:

$$\sum_{k=i}^j A[k] > 0.$$

El objetivo es implementar un algoritmo **Greedy** que calcule la **mínima cantidad de intervalos positivos** en que puede dividirse el arreglo A .

2.1. Analisis

A continuación, se presenta el análisis del algoritmo propuesto, identificando los supuestos bajo los cuales funciona, la regla Greedy que aplica y su optimalidad.

Supuestos

- El arreglo de entrada contiene números enteros, tanto positivos como negativos.
- Los intervalos de interés son segmentos de números separados por números negativos.
- La función auxiliar `suma_positivos_final` calcula, para cada posición i , la mejor suma futura (la máxima subsecuencia positiva que puede extenderse desde $i + 1$).

Limitaciones

- El algoritmo retorna la cantidad de intervalos positivos, pero no sus posiciones.
- No maneja tipos de datos distintos a enteros (ej. flotantes o nulos) dentro del arreglo de entrada.
- En casos de arreglos en los cuales no haya solución (no hay intervalos positivos), el algoritmo devuelva 0.

Regla Greedy

La regla Greedy establece que, mientras la suma acumulada del intervalo en curso más la mejor suma posible hacia adelante se mantenga positiva, se seguirá extendiendo el intervalo. En el momento en que esa condición ya no se cumple, se debe cerrar el intervalo actual y comenzar uno nuevo a partir de alguna posición siguiente, logrando así minimizar la cantidad de intervalos positivos.

En otras palabras:

- **Estado actual:** la suma parcial del intervalo en construcción (`suma_int`) y la posición del índice actual i .
- **Decisión local:** si $suma_int + pos[i] > 0$, entonces conviene seguir extendiendo, porque aún hay chances de que el intervalo se mantenga positivo. Si no existe tal posibilidad, se cierra el intervalo en el punto actual, sumando uno a la cantidad total.

Análisis de Optimalidad

El algoritmo propuesto busca minimizar la cantidad de intervalos positivos que hay dentro de un arreglo.

Para esto, sigue la siguiente idea: en cada paso se mantiene un intervalo en construcción, y se evalúa si aún es posible que dicho intervalo se mantenga positivo considerando tanto la suma acumulada actual como la mejor suma futura disponible a partir de la posición actual.

De este modo, el algoritmo evita realizar cortes tempranos, ya que únicamente cierra un intervalo cuando es imposible extenderlo sin que pierda la positividad. En consecuencia, cada intervalo construido resulta ser el más largo posible dentro de las condiciones impuestas por el arreglo.

Entonces:

- Si el algoritmo decide **cerrar un intervalo**, es porque la suma acumulada más la mejor suma futura no puede ser positiva. Esto implica que cualquier intento de extender dicho intervalo llevaría a que su suma total sea no positiva, y por lo tanto no podría considerarse válido.
- Si el algoritmo decide **extender el intervalo**, es porque aún existe la posibilidad de que la suma final sea positiva. Cerrar antes en este punto sólo generaría más intervalos, lo que lo convertiría en no óptimo.

De esta manera, cada decisión local del algoritmo es consistente con la minimización global de intervalos: nunca corta cuando todavía es posible mantener la positividad y siempre corta cuando continuar dejaría de producir un intervalo válido. Por lo tanto, el algoritmo construye la cantidad mínima de intervalos positivos, encontrando siempre la solución óptima.

2.2. Diseño

Pseudocódigo del algoritmo

```
1 FUNCION suma_positivos_final(A)
2   resultado <- diccionario
3   PARA i DESDE 0 HASTA LONGITUD(A) - 1
4     suma <- 0
5     max_suma <- -INFINITO
6     PARA j DESDE i + 1 HASTA LONGITUD(A) - 1
7       suma <- suma + A[j]
8       SI suma > max_suma ENTONCES
9         max_suma <- suma
10    FIN SI
11  FIN PARA
12  resultado[i] <- max_suma
13 FIN PARA
14 RETORNAR resultado
15 FIN FUNCION
16
17 FUNCION algoritmo_greedy(A)
18   pos <- suma_positivos_final(A)
19   suma_int <- 0
20   elems_int <- 0
21   cant_ints <- 0
22
23   PARA i DESDE 0 HASTA LONGITUD(A) - 1
24     suma_int <- suma_int + A[i]
25     elems_int <- elems_int + 1
26
27     SI elems_int <= 1 Y suma_int + pos[i] <= 0 ENTONCES
28       suma_int <- 0
29       elems_int <- 0
30       CONTINUAR
31   FIN SI
32
33   SI suma_int + pos[i] > 0 ENTONCES
```

```

34     CONTINUAR
35     FIN SI
36
37     cant_ints <- cant_ints + 1
38     suma_int <- 0
39     elems_int <- 0
40     FIN PARA
41
42     SI suma_int > 0 Y elems_int > 1 ENTONCES
43         cant_ints <- cant_ints + 1
44     FIN SI
45
46     RETORNAR cant_ints
47 FIN FUNCION

```

Estructuras de datos utilizadas

- **Diccionario (hash map):** se usa en la función `suma_positivos_final` para guardar, en cada índice i , la mejor suma futura alcanzable desde esa posición. Gracias a esto podemos acceder en tiempo constante $O(1)$ a la información de apoyo al momento de decidir si extender o cerrar un intervalo.
- **Variables escalares:** se emplean acumuladores simples (`suma_int`, `elems_int`, `cant_ints`) que permiten gestionar el estado actual del intervalo en construcción con operaciones $O(1)$.

2.3. Seguimiento del Algoritmo

- **Caso 1:** $A = [3, -5, 7, -4, 1, -8, 3, -7, 5, -9, 5, -2, 4]$, **salida esperada:** 3

Paso	x	suma_int	elems_int	pos[i]	Acción	cant_ints
0	3	3	1	2	Extender intervalo	0
1	-5	-2	2	7	Extender intervalo	0
2	7	5	3	-3	Extender intervalo	0
3	-4	1	4	1	Extender intervalo	0
4	1	2	5	-5	Cerrar intervalo	1
5	-8	-8	1	3	Ignoro número	1
6	3	3	1	-2	Extender intervalo	1
7	-7	-4	2	5	Extender intervalo	1
8	5	1	3	-2	Cerrar intervalo	2
9	-9	-9	1	7	Ignoro número	2
10	5	5	1	2	Extender intervalo	2
11	-2	3	2	4	Extender intervalo	2
12	4	7	3	0	Último intervalo válido	3

- **Caso 2:** $A = [5, -6, 5, -6, 5]$, **salida esperada:** 1

Paso	x	suma_int	elems_int	pos[i]	Acción	cant_ints
0	5	5	1	-1	Extender intervalo	0
1	-6	-1	2	5	Extender intervalo	0
2	5	4	3	-1	Extender intervalo	0
3	-6	-2	4	5	Extender intervalo	0
4	5	3	5	0	Último intervalo válido	1

- **Caso 3:** $A = [4, -6, 2, -3, -3]$, **salida esperada:** 0

Paso	x	suma_int	elems_int	pos[i]	Acción	cant_ints
0	4	4	1	-4	Ignoro número	0
1	-6	-6	1	2	Ignoro número	0
2	2	2	1	-3	Ignoro número	0
3	-3	-3	1	-3	Ignoro número	0
4	-3	-3	1	0	Ignoro número	0

- **Cerrar un intervalo:** Se cierra el intervalo actual cuando la suma acumulada `suma_int` más la mejor suma posible futura `pos[i]` es menor o igual a cero, es decir, cuando

$$\text{suma_int} + \text{pos}[i] \leq 0.$$

Además, si el intervalo tiene solo un elemento (`elems_int == 1`), **se ignora**, ya que no cumple la condición mínima de tamaño del intervalo.

- **Extender un intervalo:** Se continúa extendiendo el intervalo actual mientras que la suma acumulada `suma_int` más la mejor suma futura `pos[i]` sea estrictamente mayor que cero, es decir,

$$\text{suma_int} + \text{pos}[i] > 0.$$

Esto asegura que siempre se construye el intervalo positivo más grande posible antes de cerrarlo.

2.4. Análisis de Complejidad Temporal

Función `suma_positivos_final(arr)`

- El **bucle externo** recorre $i = 0..n - 1$, lo que da complejidad $O(n)$.
- El **bucle interno** recorre $j = i + 1..n - 1$, realizando una suma acumulativa. Sumando todas las iteraciones de i , obtenemos:

$$O(n) + O(n - 1) + O(n - 2) + \dots + O(1) = O(n^2)$$

- Por lo tanto, la complejidad de `suma_positivos_final` es cuadrática:

$$O(n^2)$$

Función `algoritmo_greedy(arr)`

- Recorre cada elemento del arreglo exactamente una vez $\Rightarrow O(n)$.
- Todas las operaciones dentro del bucle son $O(1)$, excepto la llamada inicial a `suma_positivos_final`.

Complejidad total del algoritmo: $O(n^2) + O(n) = O(n^2)$

La complejidad temporal del algoritmo es **cuadrática**, $O(n^2)$, debido al cálculo de la mejor suma futura desde cada posición. La iteración greedy sobre el arreglo tiene costo lineal y no afecta el orden de magnitud.

2.5. Set de datos

Para generar los set de datos, se implementó un generador que consiste en:

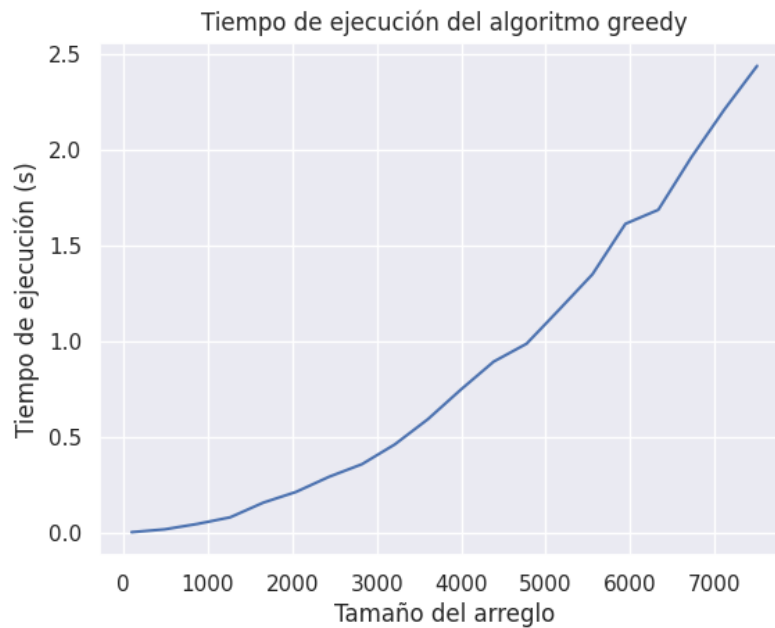
1. **Semilla fija:** Se establece una semilla aleatoria mediante `random.seed`, lo cual permite la reproducibilidad de los experimentos.
2. **Generación de arreglos:** Se construyen una cantidad (ajustable) de 20 arreglos.
 - La longitud de cada arreglo se elige de manera aleatoria dentro de un rango ajustable (por defecto, entre 0 y 10 elementos).
 - Los valores de los elementos se generan de manera uniforme en el rango $[-10, 10]$ por defecto, aun que se puede ajustar.
3. **Aplicación del algoritmo:** A cada arreglo generado se le aplica el algoritmo `algoritmo_greedy` y los resultados se almacenan en un archivo `.txt`, en el cual se muestra cada arreglo junto con el valor obtenido al aplicar el algoritmo.

2.6. Tiempos de ejecución

Se midieron los tiempos de ejecución del `algoritmo_greedy` aplicando conjuntos de datos de tamaños variables, desde 1000 hasta 7500 elementos, los cuales fueron generados a partir del generador de datos. Los resultados se presentan en los siguientes gráficos.

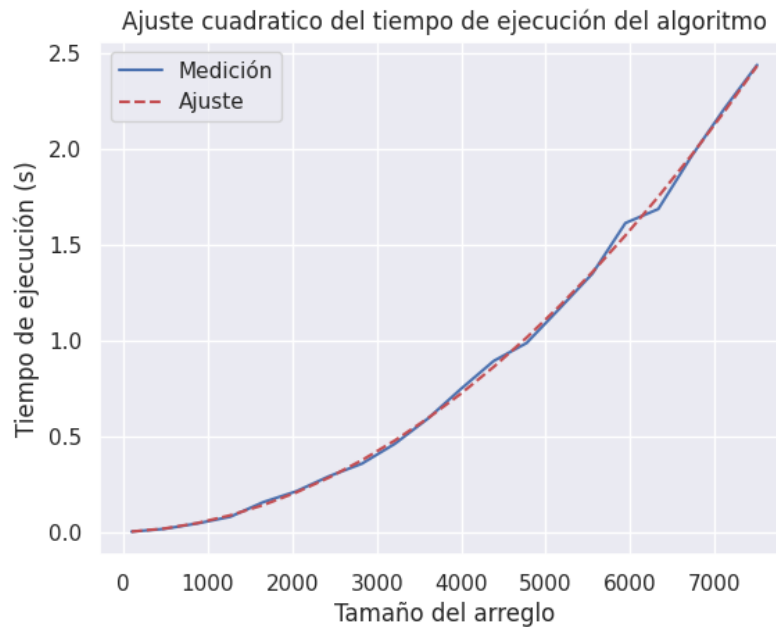
Tiempo de ejecución vs cantidad de arreglos

El primer gráfico muestra el tiempo total de ejecución del algoritmo en función de distintos arreglos variando su tamaño. Esto nos permite observar cómo aumenta el tiempo de forma cuadrática a medida que variamos la longitud del arreglo de entrada.



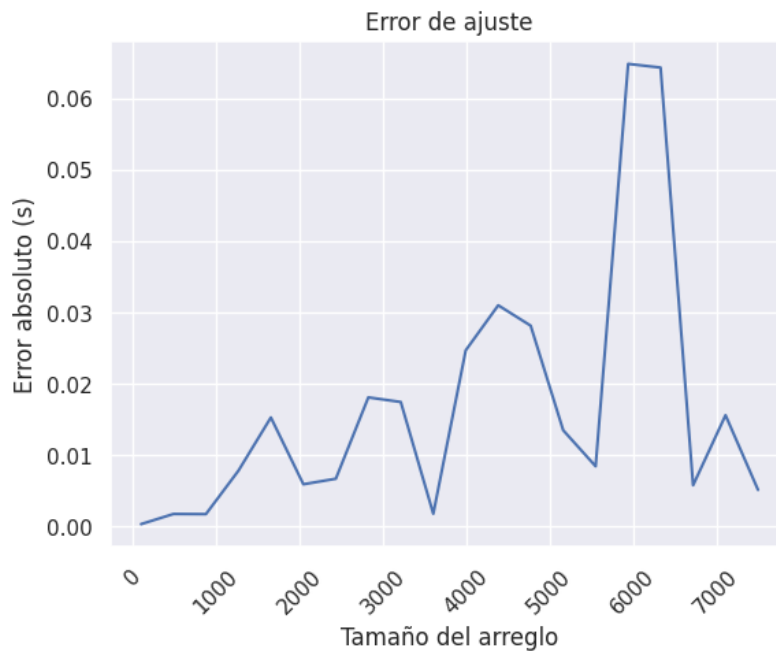
2.7. Ajuste con función cuadrática

El segundo gráfico superpone los tiempos de ejecución con una función de ajuste de la forma x^2 . Esto permite comparar visualmente la evolución real del tiempo con la predicción teórica basada en el análisis de complejidad del algoritmo.



2.8. Error cuadrático del ajuste

Finalmente, se calculó el error cuadrático total entre los tiempos medidos y los valores estimados por la función cuadrática. El error es prácticamente nulo, confirmando que el modelo cuadrático describe correctamente la complejidad temporal del algoritmo.



Observaciones

A partir de los gráficos se observa que la complejidad temporal del algoritmo crece de manera cuadrática con respecto al tamaño del arreglo de entrada. El ajuste realizado confirma que la tendencia se aproxima a un comportamiento de orden $O(n^2)$, que concuerda con el análisis previo.

3. Problema 3

El problema consiste en encontrar la secuencia de elementos contiguos dentro de un array no ordenado A cuya suma sea la máxima posible. Se solicita resolverlo mediante un algoritmo de Backtracking. El enfoque adoptado es una búsqueda exhaustiva, que explora sistemáticamente todo el espacio de soluciones factibles para garantizar la optimalidad del resultado.

3.1. Análisis

3.1.1. Supuestos, condiciones y limitaciones

- **Supuestos:** El algoritmo asume que la entrada es un array (o lista) de elementos numéricos que soportan operaciones de suma y comparación.
- **Condiciones:** Funciona con arrays no ordenados que pueden contener números positivos, negativos y/o ceros.
- **Limitaciones:** La implementación está diseñada para devolver una única subsecuencia. Si existen múltiples subsecuencias con la misma suma máxima, el algoritmo devolverá la primera que sea completamente evaluada.

3.1.2. Descripción de las podas que realiza el algoritmo

El algoritmo implementado **no realiza podas (pruning)**. Se trata de una implementación de la estrategia de backtracking en su forma más pura: una **búsqueda exhaustiva**. La estructura de bucles anidados asegura que cada posible subsecuencia contigua sea generada y evaluada.

No se aplican podas debido a que una suma parcial negativa no es un indicador fiable para descartar una rama de la búsqueda, ya que un número positivo suficientemente grande más adelante podría convertir a esa subsecuencia en la solución óptima. En otras palabras, no se puede podar caminos en relación al resultado local de nuestra suma ya que al aceptar cualquier tipo de número, ya sea positivo, negativo o cero, podríamos ramas que importan para nuestro resultado.

3.1.3. Análisis del tamaño del espacio de soluciones factibles

Una 'solución factible' es cualquier subsecuencia contigua dentro del array. Para un array de tamaño n , el número total de soluciones factibles es la suma de todas las subsecuencias de todos los largos posibles:

- Hay n subsecuencias de largo 1.
- Hay $n - 1$ subsecuencias de largo 2.
- ...
- Hay 1 subsecuencia de largo n .

El total es la suma $S = n + (n - 1) + \dots + 1$, que se calcula con la fórmula:

$$S = \frac{n(n + 1)}{2}$$

El tamaño del espacio de soluciones factibles es de orden cuadrático, es decir, $O(n^2)$.

3.2. Diseño

3.2.1. Pseudocódigo

```
1 FUNCION encontrar_maxima_subsecuencia(Array A):
2   n = longitud(A)
3
4   SI n == 0:
5     DEVOLVER (0, -1, -1)
6
7   max_suma_global = A[0]
8   mejor_inicio = 0
9   mejor_fin = 0
10
11  PARA i DESDE 0 HASTA n-1:
12    suma_actual = 0
13
14    PARA j DESDE i HASTA n-1:
15      suma_actual = suma_actual + A[j]
16
17      SI suma_actual > max_suma_global:
18        max_suma_global = suma_actual
19        mejor_inicio = i
20        mejor_fin = j
21
22  DEVOLVER (max_suma_global, mejor_inicio, mejor_fin)
```

Listing 2: Pseudocódigo del algoritmo de búsqueda exhaustiva

A continuación, se muestra el código realizado en python:

```
1
2 def subsequence_max(num_array):
3
4     if not num_array:
5         return 0, -1, -1
6
7     array_lenght = len(num_array)
8
9     max_sum_global = float('-inf')
10    best_start = -1
11    best_end = -1
12
13    for i in range(array_lenght):
14        current_sum = 0
15
16        for j in range(i, array_lenght):
17
18            current_sum += num_array[j]
19
20            # Si la suma de la subsecuencia actual (de i a j) es la mejor
21            # que hemos encontrado hasta ahora, la guardamos como la mejor solucion
22            .
23            if current_sum > max_sum_global:
24                max_sum_global = current_sum
25                best_start = i
26                best_end = j
27
28    return max_sum_global, best_start, best_end
```

Listing 3: Código de la función para hallar la secuencia de uno o más elementos contiguos que sumen el máximo valor posible

3.2.2. Estructuras de datos utilizadas

- **Array/Lista:** Se utiliza para almacenar los datos de entrada. Su elección es natural, ya que permite un acceso eficiente en tiempo constante ($O(1)$) a los elementos por su índice.

- **Tupla:** Se utilizan para almacenar `max_suma_global`, `mejor_inicio` y `mejor_fin` al momento de haber encontrado la mejor solución.

3.3. Seguimiento

Se utiliza el siguiente set de datos reducido: $A = [2, -4, 3, 1]$.

1. **Inicialización:** `max_suma_global = 2`, `mejor_inicio = 0`, `mejor_fin = 0`.
2. **$i = 0$:**
 - `j = 0`: `suma_actual = 2`. `max_suma_global` es 2.
 - `j = 1`: `suma_actual = 2 + (-4) = -2`.
 - `j = 2`: `suma_actual = -2 + 3 = 1`.
 - `j = 3`: `suma_actual = 1 + 1 = 2`.
3. **$i = 1$:**
 - `j = 1`: `suma_actual = -4`.
 - `j = 2`: `suma_actual = -4 + 3 = -1`.
 - `j = 3`: `suma_actual = -1 + 1 = 0`.
4. **$i = 2$:**
 - `j = 2`: `suma_actual = 3`. `max_suma_global` se actualiza a **3**.
 - `j = 3`: `suma_actual = 3 + 1 = 4`. `max_suma_global` se actualiza a **4**. Se guardan `mejor_inicio=2`, `mejor_fin=3`.
5. **$i = 3$:**
 - `j = 3`: `suma_actual = 1`. No mejora la suma máxima.

Resultado Final: La suma máxima es **4**, correspondiente a la subsecuencia $[3, 1]$.

3.4. Complejidad

El análisis de la complejidad temporal se realiza a partir del pseudocódigo. El bucle externo se ejecuta n veces, y el bucle interno se ejecuta en promedio $n/2$ veces. Las operaciones internas son de tiempo constante. Por lo tanto, la complejidad del algoritmo es $O(n \times n) = O(n^2)$.

Esta complejidad es consistente con el tamaño del espacio de soluciones factibles ($O(n^2)$), lo que confirma que el algoritmo realiza una exploración exhaustiva de todas las posibilidades.

3.5. Sets de Datos

Se proponen los siguientes sets de datos para verificar la correctitud del algoritmo:

- **Caso General:** $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$
Resultado: Suma = 6, Subseq = $[4, -1, 2, 1]$
- **Todos Positivos:** $[1, 2, 3, 4]$
Resultado: Suma = 10, Subseq = $[1, 2, 3, 4]$

- **Todos Negativos:** [-5, -1, -3]
Resultado: Suma = -1, Subseq = [-1]
- **Caso "Sándwich":** [-10, 4, 2, -5, 8, -20]
Resultado: Suma = 9, Subseq = [4, 2, -5, 8]

Además, hemos realizado una función para generar un array aleatorio a partir de una semilla, y determinando el min y max que puede tener un número, y su largo 'n':

```
1 import random
2
3 def generar_set_de_datos(n, semilla, min_val, max_val):
4     random.seed(semilla)
5     return [random.randint(min_val, max_val) for _ in range(n)]
```

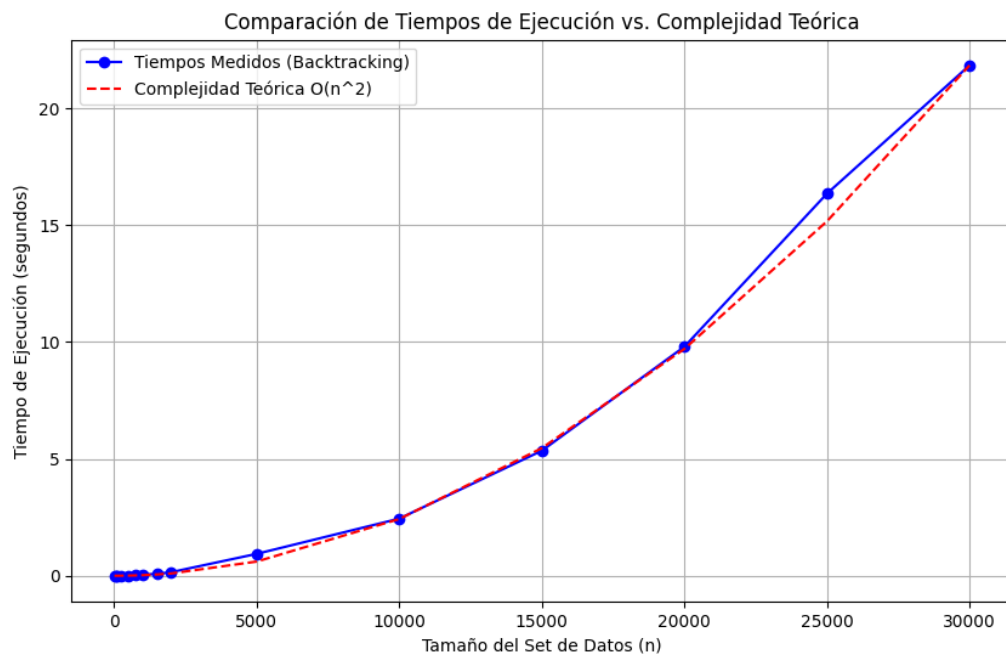
Listing 4: Código del generador de arrays aleatorios

3.6. Tiempos de ejecución

A continuación, presentamos el gráfico en donde se ve perfectamente la complejidad algorítmica $O(n^2)$.

Para este gráfico hemos seguido este test de 'stress' para ver cómo se comportaba el algoritmo con una cantidad de elementos mayor a 5000.

```
1 problem_sizes = [10, 50, 100, 250, 500, 750, 1000, 1500, 2000, 5000, 10000, 15000,
2                 20000, 25000, 30000]
3 measured_times = []
4
5 print("--- Measuring Execution Times ---")
6 for n in problem_sizes:
7     dataset = generate_dataset(n, seed=n)
8     print(f"Random dataset with {n} elements: {dataset}")
9
10    start_time = time.time()
11    result_tuple = find_max_subarray_backtracking(dataset)
12    subarray = [dataset[i] for i in range(result_tuple[1], result_tuple[2] + 1)]
13    print(f"Subarray found: {subarray} with a sum of {result_tuple[0]}")
14    end_time = time.time()
15
16    duration = end_time - start_time
17    measured_times.append(duration)
18
19    print(f"--> Time: {duration:.4f} seconds")
```



4. Problema 4

Para el Problema 4 se solicita desarrollar un algoritmo de Programación Dinámica que encuentre la secuencia de elementos contiguos que suma el máximo valor posible. El enfoque adoptado resuelve el problema mediante una iteración lineal sobre el array manteniendo sumas parciales óptimas.

4.1. Análisis

4.1.1. Supuestos, condiciones y limitaciones

- **Supuestos:** El algoritmo asume que la entrada es un array (o lista) de elementos numéricos que soportan operaciones de suma y comparación.
- **Condiciones:** Funciona con arrays no ordenados que pueden contener números positivos, negativos y/o ceros.
- **Limitaciones:** La implementación está diseñada para devolver una única subsecuencia. Si existen múltiples subsecuencias con la misma suma máxima, el algoritmo devolverá la primera que sea completamente evaluada.

4.1.2. Ecuación de Recurrencia

La ecuación de recurrencia define la suma máxima de un subarray que termina en el índice i como:

$$dp[i] = \max(A[i], dp[i-1] + A[i]) \quad (3)$$

donde $dp[i]$ representa la suma máxima del subarray que termina en i . La suma máxima global es el máximo valor en dp .

4.1.3. Justificación del cumplimiento de los requisitos de Subestructura Óptima y Subproblemas Superpuestos

- **Subestructura Óptima:** La solución óptima para un subarray que termina en i se construye a partir de la solución óptima del subarray que termina en $i-1$. Si extendemos el subarray anterior añadiendo $A[i]$, o empezamos uno nuevo en $A[i]$, garantizamos que la elección local lleva a la solución global óptima, ya que cada decisión considera la mejor suma previa.
- **Subproblemas Superpuestos:** Los subproblemas (sumas máximas terminando en cada índice) se reutilizan en la iteración. La acumulación de sumas parciales evita recalcular sumas desde cero, resolviendo subproblemas en orden lineal.

4.1.4. Indicación de cómo usa Memoization

La implementación utiliza la variable `current_sum` que actúa como memorización de la suma parcial optima anterior. Esto evita recalcular sumas previas y resuelve el problema en una sola pasada.

4.2. Diseño

4.2.1. Pseudocódigo

```
1 FUNCION encontrar_maxima_subsecuenci(Array A):
2   n = longitud(A)
3   SI n == 0:
4     DEVOLVER (0, 0, 0)
5
6   max_suma = A[0]
7   max_inicio = 0
8   max_fin = 0
9   suma_actual = A[0]
10  inicio_actual = 0
11
12  PARA i DESDE 1 HASTA n-1:
13    SI A[i] > suma_actual + A[i]:
14      suma_actual = A[i]
15      inicio_actual = i
16    SINO:
17      suma_actual = suma_actual + A[i]
18
19    SI suma_actual > max_suma:
20      max_suma = suma_actual
21      max_inicio = inicio_actual
22      max_fin = i
23
24  DEVOLVER (max_suma, max_inicio, max_fin)
```

Listing 5: Pseudocódigo del algoritmo de Programación Dinámica

A continuación, se muestra el código realizado en Python:

```
1 def subsequence_max(arr):
2     if not arr: # Manejar array vac o
3         return 0, 0, 0
4
5     # Inicializaci n con el primer elemento
6     max_sum = arr[0]
7     max_start = 0
8     max_end = 0
9     current_sum = arr[0]
10    current_start = 0
11
12    # Iterar desde el segundo elemento
13    for i in range(1, len(arr)):
14        # Decidir si empezar nuevo subarray o extender el actual
15        if arr[i] > current_sum + arr[i]:
16            current_sum = arr[i]
17            current_start = i
18        else:
19            current_sum += arr[i]
20
21    # Actualizar la soluci n global si encontramos una suma mayor
22    if current_sum > max_sum:
23        max_sum = current_sum
24        max_start = current_start
25        max_end = i
26
27    return max_sum, max_start, max_end
```

Listing 6: Código de la función para hallar la secuencia de uno o más elementos contiguos que sumen el máximo valor posible mediante Programación Dinámica

4.2.2. Estructuras de datos utilizadas

- **Array/Lista:** Se utiliza para almacenar los datos de entrada. Su elección es natural, ya que permite un acceso eficiente en tiempo constante ($O(1)$) a los elementos por su índice.
 - **Tupla:** Se utilizan para almacenar `max_suma`, `max_start` y `max_end` al momento de haber encontrado la mejor solución.
-

4.3. Seguimiento

Se utiliza el siguiente set de datos: $A = [2, -4, 3, 1]$.

Ejecución paso a paso:

1. Inicialización:

- `max_suma = A[0] = 2`
- `max_inicio = 0, max_fin = 0`
- `suma_actual = A[0] = 2`
- `inicio_actual = 0`

2. $i = 1$ ($A[1] = -4$):

- $\neg A[1] > \text{suma_actual} + A[1] \rightarrow \neg -4 > 2 + (-4) = -2 \rightarrow \text{No}$
- `suma_actual = suma_actual + A[1] = 2 + (-4) = -2`
- $\neg \text{suma_actual} > \text{max_suma} \rightarrow \neg -2 > 2 \rightarrow \text{No}$
- Estado: `max_suma = 2, suma_actual = -2, inicio_actual = 0`

3. $i = 2$ ($A[2] = 3$):

- $\neg A[2] > \text{suma_actual} + A[2] \rightarrow \neg 3 > -2 + 3 = 1 \rightarrow \text{Sí}$
- `suma_actual = A[2] = 3, inicio_actual = 2`
- $\neg \text{suma_actual} > \text{max_suma} \rightarrow \neg 3 > 2 \rightarrow \text{Sí}$
- Actualizar: `max_suma = 3, max_inicio = 2, max_fin = 2`

4. $i = 3$ ($A[3] = 1$):

- $\neg A[3] > \text{suma_actual} + A[3] \rightarrow \neg 1 > 3 + 1 = 4 \rightarrow \text{No}$
- `suma_actual = suma_actual + A[3] = 3 + 1 = 4`
- $\neg \text{suma_actual} > \text{max_suma} \rightarrow \neg 4 > 3 \rightarrow \text{Sí}$
- Actualizar: `max_suma = 4, max_inicio = 2, max_fin = 3`

Resultado Final:

- La suma máxima es 4
- Corresponde a la subsecuencia $A[2:4] = [3, 1]$ (índices 2 y 3)
- La subsecuencia va desde el índice `max_inicio = 2` hasta `max_fin = 3`

4.4. Complejidad

El análisis de la complejidad temporal se realiza a partir del pseudocódigo. Hay un único bucle que recorre el array de tamaño n , con operaciones internas de tiempo constante. Por lo tanto, la complejidad del algoritmo es $O(n)$.

4.5. Sets de Datos

Se proponen los siguientes sets de datos para verificar la correctitud del algoritmo:

- **Caso General:** [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Resultado: Suma = 6, Subseq = [4, -1, 2, 1]
- **Todos Positivos:** [1, 2, 3, 4]
Resultado: Suma = 10, Subseq = [1, 2, 3, 4]
- **Todos Negativos:** [-5, -1, -3]
Resultado: Suma = -1, Subseq = [-1]
- **Caso "Sándwich":** [-10, 4, 2, -5, 8, -20]
Resultado: Suma = 9, Subseq = [4, 2, -5, 8]

4.6. Tiempos de ejecución

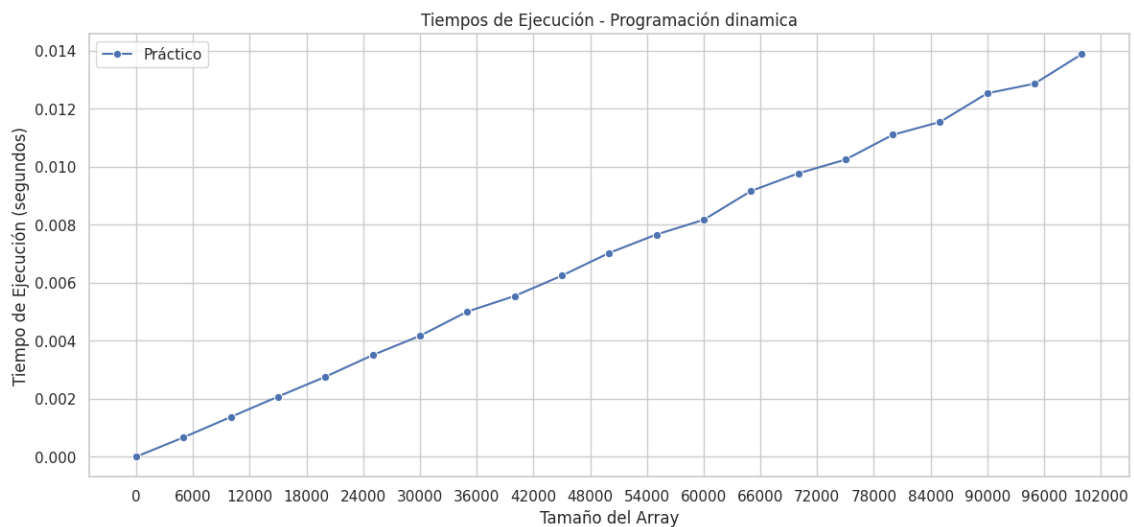
A la finalidad de reducir las variaciones debidas a factores externos —como la carga del sistema operativo, procesos en segundo plano o pequeñas fluctuaciones en el hardware— y obtener una curva más representativa de la complejidad algorítmica real, hemos iteramos múltiples veces (20) y calculamos el promedio del tiempo de ejecución para cada tamaño de entrada.

Datos obtenidos:

Tamaño Array	Tiempo de Ejecución (s)
0	0.000001
5,000	0.000621
10,000	0.001732
15,000	0.003132
20,000	0.002613
25,000	0.003101
30,000	0.003611
35,000	0.007671
40,000	0.004756
45,000	0.005519
50,000	0.006008
55,000	0.006929
60,000	0.007623
65,000	0.007881
70,000	0.008194
75,000	0.009160
80,000	0.009715
85,000	0.011269
90,000	0.010421
95,000	0.011771
100,000	0.011930

Cuadro 1: Resultados de tiempos de ejecución para distintos tamaños de array

Gráfico de datos obtenidos:

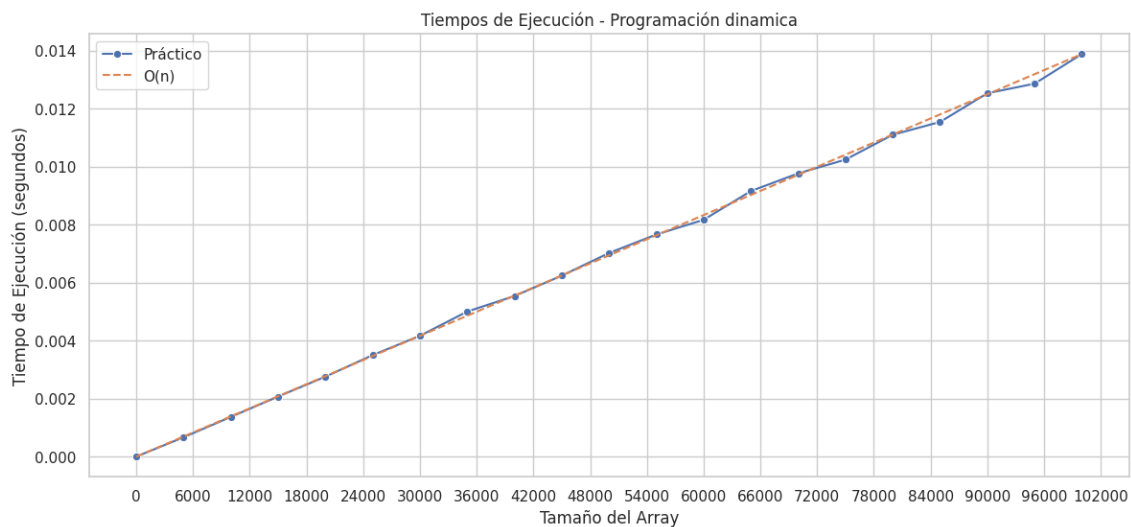


4.7. Informe de Resultados

4.7.1. Comparación de tiempos con la complejidad temporal teorica

Los tiempos de ejecución medidos para sets de datos de tamaños crecientes (0 a 100,000 elementos) muestran un crecimiento lineal, consistente con la complejidad $O(n)$. El gráfico incluye la curva de tiempos prácticos y la curva teórica $O(n)$, normalizada para coincidir con el primer punto. La curva práctica sigue de cerca la teórica permitiéndonos afirmar que se ha comportado linealmente para el set de datos utilizado.

Gráfico de datos obtenidos:



4.7.2. Comparación con el Problema 3

Los algoritmos de los Problemas 3 y 4 resuelven la misma situación: encontrar la subsecuencia contigua con suma máxima en un array de números enteros. Sin embargo, difieren significativamente en su enfoque y eficiencia.

Comparación de las complejidades computacionales

- **Problema 3 (Backtracking):** La complejidad temporal es $O(n^2)$, derivada de su búsqueda exhaustiva. Para cada posición inicial i , el algoritmo evalúa todas las subsecuencias que comienzan en i hasta el final del array, resultando en un número total de operaciones proporcional a la suma de los primeros n enteros, $\frac{n(n+1)}{2}$, que es del orden de $O(n^2)$.

- **Problema 4 (Programación Dinámica):** La complejidad temporal es $O(n)$, ya que realiza una sola iteración sobre el array, con operaciones constantes por elemento.

Teóricamente, el algoritmo de Programación Dinámica debería ser más eficiente, ya que su complejidad lineal $O(n)$ crece mucho más lentamente que la cuadrática $O(n^2)$ del Backtracking a medida que aumenta el tamaño del array. Esta diferencia se amplifica en arrays grandes, donde el costo computacional del Backtracking se vuelve prohibitivo.

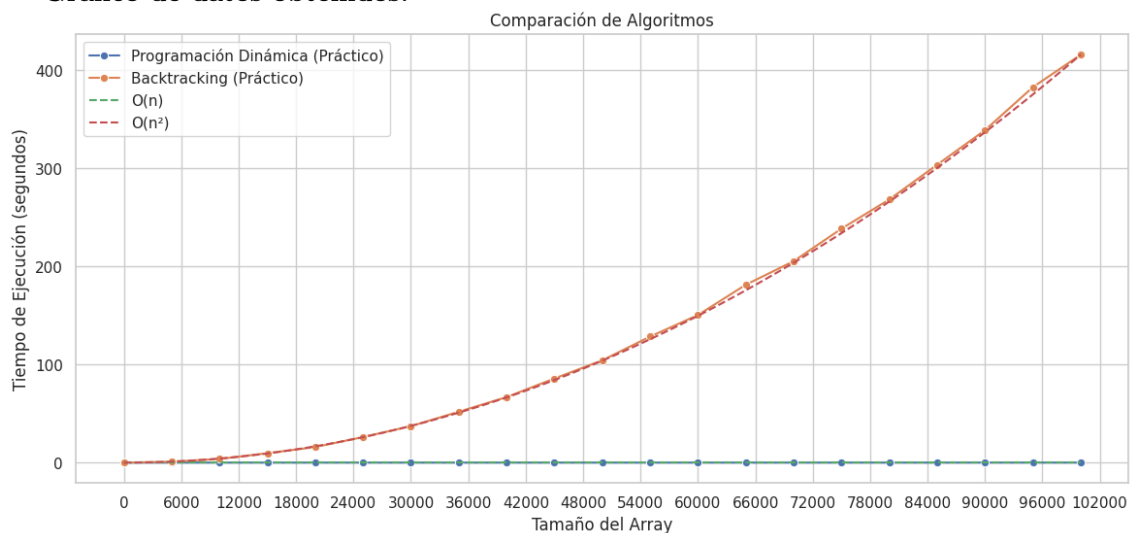
Para realizar la comparación del algoritmo desarrollado con estrategia de backtracking y el desarrollado con programación dinámica ampliamos las pruebas de backtracking hasta un array de 100.000 números.

Datos obtenidos:

Tamaño Array	Backtracking tiempos (s)	Programación dinámica tiempos (s)
0	0.000002	0.000001
5,000	0.957368	0.000621
10,000	3.749397	0.001732
15,000	9.613987	0.003132
20,000	15.884037	0.002613
25,000	26.083079	0.003101
30,000	37.190171	0.003611
35,000	51.901774	0.007671
40,000	66.804682	0.004756
45,000	85.791855	0.005519
50,000	104.298462	0.006008
55,000	128.547472	0.006929
60,000	150.525165	0.007623
65,000	181.409610	0.007881
70,000	205.418513	0.008194
75,000	238.576020	0.009160
80,000	268.388445	0.009715
85,000	303.800514	0.011269
90,000	339.048621	0.010421
95,000	382.862299	0.011771
100,000	415.985524	0.011930

Cuadro 2: Resultados de tiempos de ejecución para distintos tamaños de array

Gráfico de datos obtenidos:



La curva práctica del Backtracking sigue de cerca la teórica $O(n^2)$, mostrando un crecimiento exponencialmente más rápido que la curva lineal $O(n)$ de la Programación Dinámica. El gráfico confirma que, a partir de tamaños moderados (alrededor de 5000 elementos), la diferencia en rendimiento se vuelve significativa, haciendo del algoritmo de programación dinámica la elección preferida para arrays grandes.